

Q1. [EASY] Reverse a Number

Problem Statement

Given an integer n , write a function that returns the number obtained by reversing its digits.

```
In [3]: def reverse_number(n):
        reversed_num = 0

        while n > 0:
            digit = n % 10          # Extract last digit
            reversed_num = reversed_num * 10 + digit
            n //= 10               # Remove last digit

        return reversed_num
        print(reverse_number(1234))
        reverse_number(1234)
```

Out[3]: 4321

```
In [4]: def reverse_number(n):
        reversed_num = 0

        while n > 0:
            digit = n % 10          # Extract last digit
            reversed_num = reversed_num * 10 + digit
            n //= 10               # Remove last digit

        return reversed_num
        print(reverse_number(980))
        reverse_number(980)
```

Out[4]: 89

Q2. [EASY] Palindrome Number

Problem Statement

Given an integer n , return True if it reads the same from left to right and right to left, otherwise return False.

```
In [5]: def is_palindrome(n):
        original = n
        reversed_num = 0

        while n > 0:
            digit = n % 10
            reversed_num = reversed_num * 10 + digit
            n //= 10

        return original == reversed_num
        is_palindrome(121)
```

Out[5]: True

```
In [6]: def is_palindrome(n):
        original = n
        reversed_num = 0

        while n > 0:
            digit = n % 10
            reversed_num = reversed_num * 10 + digit
            n //= 10

        return original == reversed_num
is_palindrome(123)
```

Out[6]: False

Q3. [EASY] Function Without Arguments

Problem Statement

Write a function greet() that takes NO parameters and prints a welcome message.

```
In [12]: def greet():
        print("Hello! Welcome to python funtin")
greet()
```

Hello! Welcome to python funtin

Q4. [EASY] Largest of Three Numbers

Problem Statement

Write a function largest_of_three(a, b, c) that returns the largest of three given numbers using only if/elif/else.

```
In [15]: def largest_of_three(a, b, c):
        if a >= b and a >= c:
            return a
        elif b >= a and b >= c:
            return b
        else:
            return c
largest_of_three(10,25,15)
```

Out[15]: 25

```
In [16]: def largest_of_three(a, b, c):
        if a >= b and a >= c:
            return a
        elif b >= a and b >= c:
            return b
        else:
            return c
largest_of_three(7,7,3)
```

Out[16]: 7

Q5. [EASY] Function With Default Arguments

Problem Statement

Write a function `greet_user(name="Guest")` that has a default parameter and prints a personalized greeting message.

```
In [18]: def greet_user(name="Guest"):
          print(f"Hello, {name}! Welcome!")
          greet_user()
```

Hello, Guest! Welcome!

```
In [19]: def greet_user(name="Guest"):
          print(f"Hello, {name}! Welcome!")
          greet_user("Sana Swathi Reddy")
```

Hello, Sana Swathi Reddy! Welcome!

```
In [20]: def greet_user(name="Guest"):
          print(f"Hello, {name}! Welcome!")
          greet_user("Arun Reddy")
```

Hello, Arun Reddy! Welcome!

Q6. [EASY] Swap Without Temp Variable

Problem Statement

Write a function `swap_no_temp(a, b)` that swaps two values **WITHOUT** using a third temporary variable, and returns both.

```
In [23]: def swap_no_temp(a, b):
          a, b = b, a
          return a, b
          swap_no_temp(5,10)
```

Out[23]: (10, 5)

```
In [24]: def swap_no_temp(a, b):
          a, b = b, a
          return a, b
          swap_no_temp(100,200)
```

Out[24]: (200, 100)

Q7. [EASY] Check if Number is Positive, Negative, or Zero

Problem Statement

Write a function `check_sign(n)` that takes an integer `n` and returns "Positive" if `n > 0`, "Negative" if `n < 0`, and "Zero" if `n == 0`.

```
In [27]: def check_sign(n):
          if n > 0:
              return "Positive"
          elif n < 0:
              return "Negative"
          else:
```

```

        return "Zero"
    check_sign(42)

```

Out[27]: 'Positive'

```

In [28]: def check_sign(n):
        if n > 0:
            return "Positive"
        elif n < 0:
            return "Negative"
        else:
            return "Zero"
    check_sign(-7)

```

Out[28]: 'Negative'

```

In [29]: def check_sign(n):
        if n > 0:
            return "Positive"
        elif n < 0:
            return "Negative"
        else:
            return "Zero"
    check_sign(0)

```

Out[29]: 'Zero'

Q8. [EASY] Function That Prints vs Function That Returns

Problem Statement

Write TWO functions: `say_hello()` that prints "Hello!" but does NOT return anything, and `make_hello()` that returns the string "Hello!". Then compare their outputs.

```

In [36]: def say_hello():
        print("Hello")

a=say_hello()
print("a=",a)

```

Hello
a= None

```

In [41]: def make_hello():
        return "Hello!"

b=make_hello()
print("b=",b)

```

b= Hello!

Q9. [EASY] Random Even or Odd

Problem Statement

****Write a function `even_or_odd_random()` with NO parameters. It generates a random number between 1 and 100 using the random module, and returns the number along**

with whether it is 'even' or 'odd'.***

```
In [42]: import random

def even_or_odd_random():
    number = random.randint(1, 100)

    if number % 2 == 0:
        return number, "even"
    else:
        return number, "odd"
even_or_odd_random()
```

Out[42]: (54, 'even')

```
In [43]: import random

def even_or_odd_random():
    number = random.randint(1, 100)

    if number % 2 == 0:
        return number, "even"
    else:
        return number, "odd"
even_or_odd_random()
```

Out[43]: (87, 'odd')

****Q10. [EASY] Circle Area Using Math Module ****

Problem Statement

Write a function `circle_area(r)` that uses the math module to calculate the area of a circle with radius `r`. Return the area rounded to 2 decimals.

```
In [3]: import math

def circle_area(r):
    return round(math.pi * r * r, 2)
circle_area(5)
```

Out[3]: 78.54

```
In [4]: import math

def circle_area(r):
    return round(math.pi * r * r, 2)
circle_area(10)
```

Out[4]: 314.16

```
In [5]: import math

def circle_area(r):
    return round(math.pi * r * r, 2)
circle_area(7)
```

Out[5]: 153.94

Q11. [MEDIUM] Positional vs Keyword Arguments

Problem Statement

Write a function `describe_pet(animal, name, age)` that prints: "I have a -year-old named ." Then call it in four different ways to demonstrate positional and keyword arguments.

```
In [6]: def describe_pet(animal, name, age):  
        print(f"I have a {age}-year-old {animal} named {name}.")  
  
        # 1. All positional arguments  
        describe_pet("Dog", "Buddy", 3)
```

I have a 3-year-old Dog named Buddy.

```
In [7]: def describe_pet(animal, name, age):  
        print(f"I have a {age}-year-old {animal} named {name}.")  
        describe_pet(animal="Cat", name="Whiskers", age=2)
```

I have a 2-year-old Cat named Whiskers.

```
In [8]: def describe_pet(animal, name, age):  
        print(f"I have a {age}-year-old {animal} named {name}.")  
        describe_pet("Parrot", name="Coco", age=1)
```

I have a 1-year-old Parrot named Coco.

```
In [9]: def describe_pet(animal, name, age):  
        print(f"I have a {age}-year-old {animal} named {name}.")  
        describe_pet(age=5, name="Max", animal="Rabbit")
```

I have a 5-year-old Rabbit named Max.

Q12. [MEDIUM] Global vs Local Scope

Problem Statement

Write a program that demonstrates the difference between global and local variables. Define a global variable `score = 100`. Inside a function `update_score(value)`, use the global `score`, add a local `bonus = 50`, and compute `new_score`. Show that `bonus` is not accessible outside the function.

```
In [21]: score = 100  
  
def update_score(value):  
    global score  
    bonus = 50          # Local variable  
    new_score = score + value + bonus  
  
    print("Inside score: new_score =", new_score)  
    print("Inside score : bonus =", bonus)  
  
update_score(25)
```

```
Inside score: new_score = 175
Inside score : bonus = 50
```

```
In [10]: score=100
def outside_score(value):
    print("outside_score:",score)
    outside_score(100)
```

```
outside_score: 100
```

```
In [28]: def outside_bonus(bonus):
        print("Outside: bonus =", bonus)

        outside_bonus(bonus)
        print("Outside: bonus =", bonus)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[28], line 4
      1 def outside_bonus(bonus):
      2     print("Outside: bonus =", bonus)
----> 4 outside_bonus(bonus)
      5 print("Outside: bonus =", bonus)

NameError: name 'bonus' is not defined
```

```
In [43]: def update_score():
        try:
            print("Outside: bonus =", bonus)
        except Exception as e:
            print(e)
        update_score()
```

```
name 'bonus' is not defined
```

Q13. [MEDIUM] Try-Except Inside a Function

Problem Statement

Write a function `safe_divide(a, b)` that safely divides two numbers using `try/except`. Catch `ZeroDivisionError` and `TypeError` with appropriate messages.

```
In [44]: def safe_divide(a, b):
        try:
            result = a / b
            print("Result =", result)
        except ZeroDivisionError:
            print("Error: Cannot divide by zero.")
        except TypeError:
            print("Error: Both inputs must be numbers.")

        safe_divide(10, 2)
```

```
Result = 5.0
```

```
In [45]: def safe_divide(a, b):
        try:
            result = a / b
            print("Result =", result)
        except ZeroDivisionError:
```

```

        print("Error: Cannot divide by zero.")
    except TypeError:
        print("Error: invaild input.")

safe_divide(10, 0)

```

Error: Cannot divide by zero.

```

In [48]: def safe_divide(a, b):
        try:
            result = a / b
            print("Result =", result)
        except ZeroDivisionError:
            print("Error: Cannot divide by zero.")
        except TypeError:
            print("Error: invaild input type.")

safe_divide(10, "hello")

```

Error: invaild input type.

```

In [50]: def safe_divide(a, b):
        try:
            result = a / b
            print("Result =", result)
        except ZeroDivisionError:
            print("Error: Cannot divide by zero.")
        except TypeError:
            print("Error: invaild input type.")

safe_divide("abc", 2)

```

Error: invaild input type.

Q14. [MEDIUM] Reverse Words in a Sentence

Problem Statement

Write a function `reverse_words(sentence)` that reverses the ORDER of words (not the letters) in a sentence.

```

In [52]: def reverse_words(sentence):
        print(sentence[::-1])
        reverse_words("Python is easy")

```

ysae si nohtyP

```

In [53]: def reverse_words(sentence):
        words = sentence.split()
        for i in range(len(words)-1, -1, -1):
            print(words[i], end=" ")

reverse_words("Python is easy")

```

easy is Python

In []: